

i need to make a python functions for my app ,
The city wants to improve its public transport system so that it is easier for people to use buses and trains. Different types of passengers use the system, such as students, seniors, and regular commuters, and they may have different needs when buying tickets.

Passengers should be able to buy tickets using a mobile app or another digital service. They can choose different types of tickets, such as single rides, time-based passes, or ticket bundles.

When a passenger selects a ticket, the system should automatically calculate the correct price based on the passenger's category and the type of ticket they choose. For example, students may get a discount, while seniors may have a different price reduction.

Passengers may also buy more than one ticket at a time, and the system should handle these purchases correctly even if there are different combinations of tickets. If a passenger enters something incorrect, such as not choosing a valid number of tickets, the system should handle it in a clear and safe way.

The system should always calculate prices correctly, even when many people are using it at the same time during busy periods.

The system should also keep passengers' personal details and payment information safe and protected.

Show less 

A clean way to design this in Python is to separate **passenger types**, **ticket types**, and a **pricing engine**. That keeps it easy to extend and safe for concurrency later.

Below is a solid starting structure with functions + basic safety checks.



Core Design Idea

We'll model:

- Passenger types → discount rules
- Ticket types → base price logic
- Purchase system → validation + total calculation



Python Implementation

Python

Run

```
from dataclasses import dataclass
from enum import Enum
from typing import List
```

```
# -----
# Passenger Categories
# -----
class PassengerType(Enum):
    REGULAR = "regular"
    STUDENT = "student"
    SENIOR = "senior"
```

```
# -----
# Ticket Types
# -----
class TicketType(Enum):
    SINGLE = "single"
    PASS_24H = "24h_pass"
    BUNDLE_10 = "bundle_10"
```

```
# -----
# Ticket Prices (base system)
# -----
TICKET_PRICES = {
    TicketType.SINGLE: 3.0,
    TicketType.PASS_24H: 10.0,
    TicketType.BUNDLE_10: 25.0,
}
```

```
# -----
# Discount Rules
# -----
DISCOUNTS = {
```

```

        PassengerType.REGULAR: 0.0,
        PassengerType.STUDENT: 0.30, # 30% off
        PassengerType.SENIOR: 0.40, # 40% off
    }

# -----
# Passenger Model
# -----
@dataclass
class Passenger:
    name: str
    passenger_type: PassengerType

# -----
# Validation
# -----
def validate_quantity(quantity: int):
    if not isinstance(quantity, int):
        raise ValueError("Quantity must be an integer.")
    if quantity <= 0:
        raise ValueError("Quantity must be greater than 0.")

# -----
# Price Calculation
# -----
def calculate_ticket_price(
    passenger: Passenger,
    ticket_type: TicketType,
    quantity: int = 1
) -> float:

    validate_quantity(quantity)

    base_price = TICKET_PRICES[ticket_type]
    discount = DISCOUNTS[passenger.passenger_type]

    total = base_price * quantity
    discounted_total = total * (1 - discount)

    return round(discounted_total, 2)

# -----
# Multiple Ticket Purchase
# -----
def purchase_tickets(
    passenger: Passenger,
    orders: List[tuple]
) -> float:

```

```
"""
orders format:
[(TicketType.SINGLE, 2), (TicketType.PASS_24H, 1)]
"""

if not orders:
    raise ValueError("No tickets selected.")

total_cost = 0.0

for ticket_type, qty in orders:
    if ticket_type not in TicketType:
        raise ValueError(f"Invalid ticket type: {ticket_type}")

    total_cost += calculate_ticket_price(passenger, ticket_type, qty)

return round(total_cost, 2)
```



Example Usage

Python

Run

```
user = Passenger("Nikan", PassengerType.STUDENT)

orders = [
    (TicketType.SINGLE, 2),
    (TicketType.PASS_24H, 1)
]

total = purchase_tickets(user, orders)

print("Total price:", total)
```



About Safety & Real Apps

To match your requirements about protection and concurrency:

1. Thread safety (important for busy systems)

In real systems:

- Use database transactions
- Or lock critical payment sections
- Or use backend services (FastAPI + DB)

Python example idea:

Python

Run

```
import threading  
lock = threading.Lock()
```

2. Security (very important)

Never store raw payment info. Instead:

- Use payment provider tokens (Stripe, Adyen, etc.)
- Encrypt personal data
- Hash sensitive fields



If you want to improve it further

I can help you extend this into:

- FastAPI backend (real API system)
- Database model (PostgreSQL)
- User login + JWT authentication
- QR-code ticket system
- Real-time ticket validation

Just tell me 👍

        Sources